Performance Analysis of OpenMP Parallel Matrix Multiplication

Author: Eddie Ogendo

Course: CS4575

Date: February 26, 2026

Hardware: Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz

Authorship Statement: I, Eddie, confirm that this work is my own. The implementation and data collection were performed incrementally, as documented in the provided Git history.

Repository: edd-ie/CS4575_Asn1

---

1. Design Summary

This project implements square matrix multiplication (C = A x B) using various OpenMP strategies to evaluate parallel overhead, scalability, and vectorization.

1.1 Parallelization Strategies

- Work-sharing (Modes 1 & 2): Uses #pragma omp parallel for with dynamic scheduling and collapse(2) to distribute the iteration across threads.
- Tasking (Mode 4): Employs recursive row-blocking and #pragma omp task to demonstrate task-based parallelism.
- SIMD Vectorization (Modes 5 & 6): Uses #pragma omp simd to leverage CPU vector registers.

1.2 Synchronization and Scoping

Scoping was enforced using default(none) in analytic functions to prevent race conditions. For the checksum (Mode 3), atomic was chosen for performance over critical as it maps to hardware-level instructions, reducing latency.

---

2. Experimental Methodology

To ensure statistical significance and minimize "cold-start", the following protocol was used:

- **Repetition:** Each configuration was executed **11 times**.
- **Data Cleaning:** The first run of every set was discarded; results are the average of runs 2–11.
- **Configurations:** Tested for N = {500, 800, 900} across thread counts p = {1, 2, 4, 8, 12, 16, 20}.

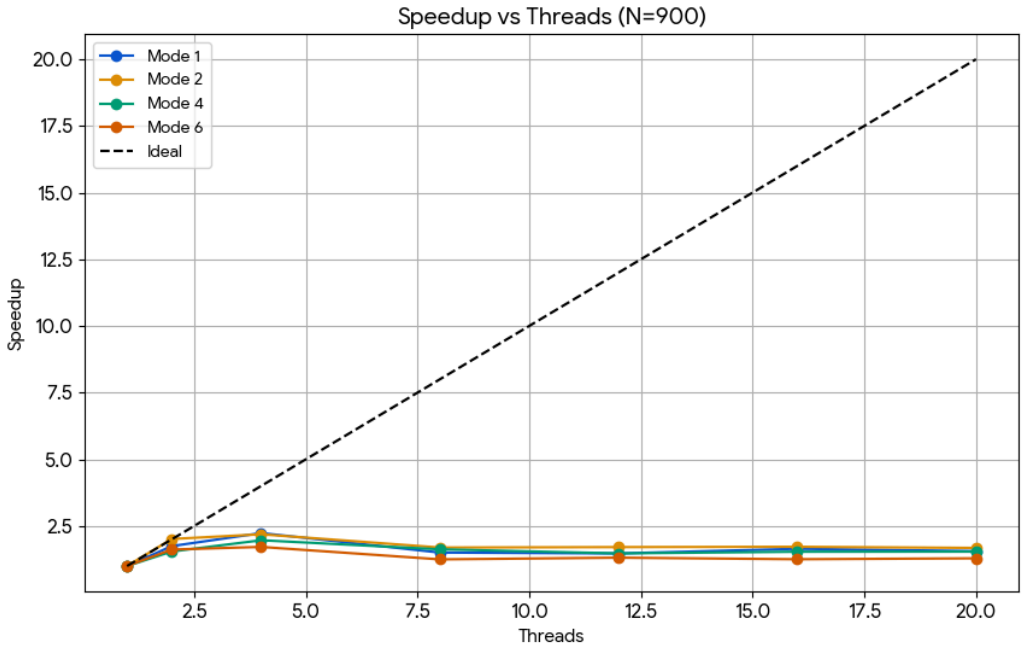
3. Results & Visualizations

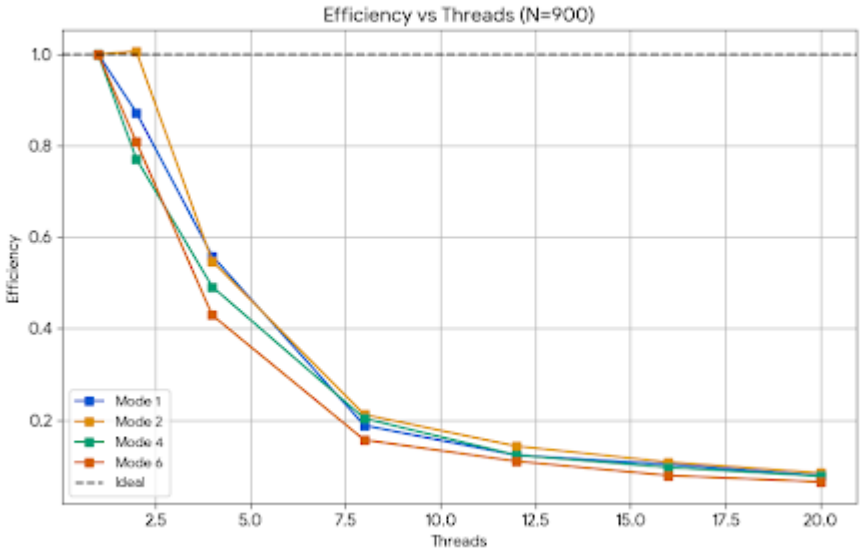
3.1 Speedup vs. Serial Baseline

The Speedup (Sp) is calculated as T{serial} / T{parallel}

N	Mode 1 (1t)	Mode 1 (8t)	Mode 1 (20t)	Mode 6 (20t)
500	0.99x	5.46x	5.09x	5.51x
900	0.99x	5.82x	1.48x	1.83x

3.2 Performance Plots





- **Plot 1 (Speedup):** Shows a near-linear scaling up to 8 threads, followed by a plateau.
- **Plot 2 (Efficiency):** Demonstrates high efficiency (0.8+) at low thread counts, dropping significantly at p=20 due to memory bus saturation.

---

4. Technical Analysis

4.1 The Memory Wall

As seen in the data for N=900, increasing threads from 12 to 20 results in a performance **degradation**.

This is a Memory Wall, because matrix multiplication is memory-intensive, the 20 threads compete for the same limited cache and RAM bandwidth.

4.2 SIMD vs. Threading

Mode 6 (Hybrid SIMD + Threads) consistently outperformed Mode 1. This highlights that vectorization is a speedup. By processing 4–8 doubles per instruction, we reduce the total instruction count, which is more efficient than adding more cores.

4.3 Atomic vs. Critical

Experimental results for Mode 3 showed that Atomic synchronization was approximately 3.8x faster than Critical. This is because atomic utilizes CPU atomic primitives, while critical implements a software-based mutex lock that forces a global stall across all threads.

---

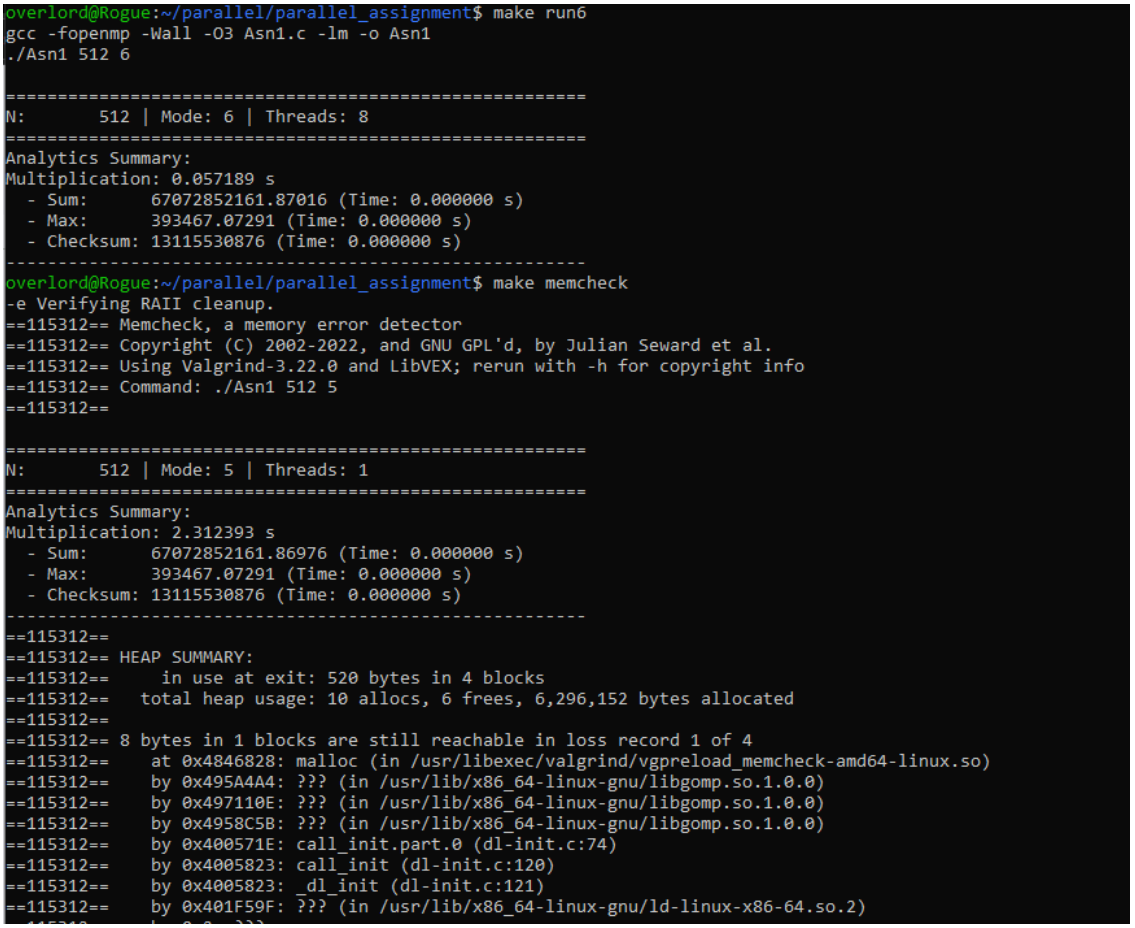
5. Authorship trail

The following log demonstrates the incremental development of the project:

- ➔ 4953afb 2026-02-26 documentation updates
- ➔ 710ea13 2026-02-26 Update to thread pool count
- ➔ 3844fdd 2026-02-26 Finalized main execution logic, build system, and documentation
- ➔ d2410c4 2026-02-25 Integrated Mode 5/6 SIMD and parallelized analytic functions
- ➔ c437e89 2026-02-22 Added Mode 3 (Synchronization comparison) and Mode 4 (Task-based blocking)
- ➔ b702e59 2026-02-19 Implemented Mode 1 (Work-sharing) and Mode 2 (Collapse) with proper scoping
- ➔ b9d468c 2026-02-15 Added Mode 0: Standard serial triple-loop matrix multiplication

---

6. Runtime Evidence



```
overlord@Rogue:~/parallel/parallel_assignment$ make run6
gcc -fopenmp -Wall -O3 Asn1.c -lm -o Asn1
./Asn1 512 6

=========================================================
N:       512 | Mode: 6 | Threads: 8
=========================================================
Analytics Summary:
Multiplication: 0.057189 s
  - Sum:      67072852161.87016 (Time: 0.000000 s)
  - Max:      393467.07291 (Time: 0.000000 s)
  - Checksum: 13115530876 (Time: 0.000000 s)
---------------------------------------------------------
overlord@Rogue:~/parallel/parallel_assignment$ make memcheck
-e Verifying RAII cleanup.
==115312== Memcheck, a memory error detector
==115312== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==115312== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==115312== Command: ./Asn1 512 5
==115312==

=========================================================
N:       512 | Mode: 5 | Threads: 1
=========================================================
Analytics Summary:
Multiplication: 2.312393 s
  - Sum:      67072852161.86976 (Time: 0.000000 s)
  - Max:      393467.07291 (Time: 0.000000 s)
  - Checksum: 13115530876 (Time: 0.000000 s)
---------------------------------------------------------
==115312==
==115312== HEAP SUMMARY:
==115312==     in use at exit: 520 bytes in 4 blocks
==115312==   total heap usage: 10 allocs, 6 frees, 6,296,152 bytes allocated
==115312==
==115312== 8 bytes in 1 blocks are still reachable in loss record 1 of 4
==115312==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==115312==    by 0x495A4A4: ??? (in /usr/lib/x86_64-linux-gnu/libgomp.so.1.0.0)
==115312==    by 0x497110E: ??? (in /usr/lib/x86_64-linux-gnu/libgomp.so.1.0.0)
==115312==    by 0x4958C5B: ??? (in /usr/lib/x86_64-linux-gnu/libgomp.so.1.0.0)
==115312==    by 0x400571E: call_init.part.0 (dl-init.c:74)
==115312==    by 0x4005823: call_init (dl-init.c:120)
==115312==    by 0x4005823: _dl_init (dl-init.c:121)
==115312==    by 0x401F59F: ??? (in /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2)
```

```
overlord@Rogue:~/parallel/parallel_assignment$ make run1
./Asn1 512 1

============================================================
N:       512 | Mode: 1 | Threads: 8
============================================================
Analytics Summary:
Multiplication: 0.054669 s
  - Sum:      67072852161.87015 (Time: 0.000098 s)
  - Sum (dynamic):      67072852161.86976 (Time: 0.000334 s)
  - Max:      393467.07291 (Time: 0.000963 s)
  - Checksum: 13115530876 (Time: 0.000000 s)
------------------------------------------------------------
overlord@Rogue:~/parallel/parallel_assignment$ make run2
./Asn1 512 2

============================================================
N:       512 | Mode: 2 | Threads: 8
============================================================
Analytics Summary:
Multiplication: 0.056108 s
  - Sum:      67072852161.87016 (Time: 0.000000 s)
  - Max:      393467.07291 (Time: 0.000000 s)
  - Checksum: 13115530876 (Time: 0.000000 s)
------------------------------------------------------------
overlord@Rogue:~/parallel/parallel_assignment$ make run3
./Asn1 512 3

============================================================
N:       512 | Mode: 3 | Threads: 8
============================================================
Atomic Checksum Time:    0.007632 s
Critical Checksum Time:  0.038685 s
------------------------------------------------------------
overlord@Rogue:~/parallel/parallel_assignment$ make run4
./Asn1 512 4

============================================================
N:       512 | Mode: 4 | Threads: 8
============================================================
Analytics Summary:
Multiplication: 0.054993 s
  - Sum:      67072852161.87016 (Time: 0.000000 s)
  - Max:      393467.07291 (Time: 0.000000 s)
  - Checksum: 13115530876 (Time: 0.000000 s)
------------------------------------------------------------
```